Underactuated Robotics

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024
Last modified 2026-10-11.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter Table of contents Next Chapter

Chapter 3
Acrobots, Cart-Poles, and Quadrotors

A great deal of work in the control of underactuated systems has been done in the context of low-dimensional model systems. These model systems capture the essence of the problem without introducing all of the complexity that is often involved in more real-world examples. In this chapter we will focus on two of the most well-known and well-studied model systems--the Acrobot and the Cart-Pole. After we have developed some tools, we will see that they can be applied directly to other model systems; we will give a number of examples using Quadrotors. All of these systems are trivially underactuated, having fewer actuators than degrees of freedom.

3.1 The Acrobot

The Acrobot is a planar two-link robotic arm in the vertical plane (working against gravity), with an actuator at the elbow, but no actuator at the shoulder. It was first described in detail in [1]. The companion system, with an actuator at the shoulder but not at the elbow, is known as the Pendubot[2]. The Acrobot is so named because of its resemblance to a gymnast (or acrobat) on a parallel bar, who controls his/her motion predominantly by effort at the waist (and not effort at the wrist). The most common control task studied for the Acrobot is the swing-up task, in which the system must use the elbow (or waist) torque to move the system into a vertical configuration then balance.

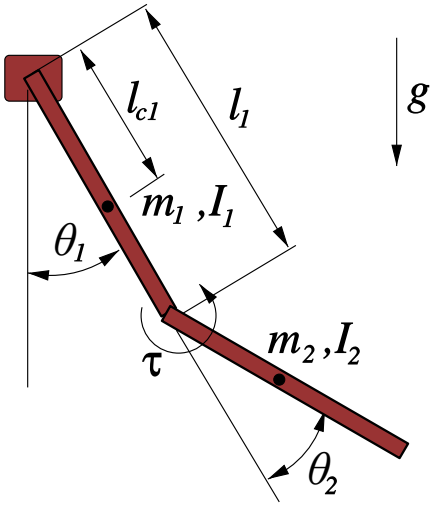


Figure 3.1 – The Acrobot. Click here to see a physical Acrobot swing up and balance.

The Acrobot is representative of the primary challenge in underactuated robots. In order to swing up and balance the entire system, the controller must reason about and exploit the state–dependent coupling between the actuated degree of freedom and the unactuated degree of freedom. It is also an important system because, as we will see, it closely resembles one of the simplest models of a walking robot.

3.1.1 Equations of motion

Figure 3.1 illustrates the model parameters used in our analysis. θ_1 is the shoulder joint angle, θ_2 is the elbow (relative) joint angle, and we will use $\mathbf{q} = [\theta_1, \theta_2]^T$, $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T$. The zero configuration is with both links pointed directly down. The moments of inertia, I_1, I_2 are taken about the pivots. The task is to stabilize the unstable fixed point $\mathbf{x} = [\pi, 0, 0, 0]^T$.

We will derive the equations of motion for the Acrobot using the method of Lagrange. The locations of the center of mass of each link, $\mathbf{p}_{c1}, \mathbf{p}_{c2}$, are given by the kinematics:

$$\mathbf{p}_{c1} = \begin{bmatrix} l_{c1} s_1 \\ -l_{c1} c_1 \end{bmatrix}, \quad \mathbf{p}_{c2} = \begin{bmatrix} l_1 s_1 + l_{c2} s_{1+2} \\ -l_1 c_1 - l_{c2} c_{1+2} \end{bmatrix}, \tag{1}$$

where s_1 is shorthand for $\sin(\theta_1)$, c_{1+2} is shorthand for $\cos(\theta_1 + \theta_2)$, etc. The energy is given by:

$$T = T_1 + T_2, \quad T_1 = \frac{1}{2} I_1 \dot{q}_1^2 \tag{2}$$

$$T_2 = \frac{1}{2}(m_2 l_1^2 + I_2 + 2 m_2 l_1 l_{c2} c_2)\dot{q}_1^2 + \frac{1}{2} I_2 \dot{q}_2^2 + (I_2 + m_2 l_1 l_{c2} c_2)\dot{q}_1 \dot{q}_2 \tag{3}$$

$$U = -m_1 g l_{c1} c_1 - m_2 g (l_1 c_1 + l_{c2} c_{1+2}) \tag{4}$$

Entering these quantities into the Lagrangian yields the equations of motion:

$$(I_1 + I_2 + m_2 l_1^2 + 2 m_2 l_1 l_{c2} c_2)\ddot{q}_1 + (I_2 + m_2 l_1 l_{c2} c_2)\ddot{q}_2 - 2 m_2 l_1 l_{c2} s_2 \dot{q}_1 \dot{q}_2 \tag{5}$$

$$-m_2 l_1 l_{c2} s_2 \dot{q}_2^2 + m_1 g l_{c1} s_1 + m_2 g (l_1 s_1 + l_{c2} s_{1+2}) = 0 \tag{6}$$

$$(I_2 + m_2 l_1 l_{c2} c_2)\ddot{q}_1 + I_2 \ddot{q}_2 + m_2 l_1 l_{c2} s_2 \dot{q}_1^2 + m_2 g l_{c2} s_{1+2} = \tau \tag{7}$$

In standard, manipulator equation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

using $\mathbf{q} = [\theta_1, \theta_2]^T$, $\mathbf{u} = \tau$ we have:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} I_1 + I_2 + m_2 l_1^2 + 2m_2 l_1 l_{c2} c_2 & I_2 + m_2 l_1 l_{c2} c_2 \\ I_2 + m_2 l_1 l_{c2} c_2 & I_2 \end{bmatrix}, \tag{8}$$

$$\mathbf{C}(\mathbf{q},\dot{\mathbf{q}}) = \begin{bmatrix} -2m_2 l_1 l_{c2} s_2 \dot{q}_2 & -m_2 l_1 l_{c2} s_2 \dot{q}_2 \\ m_2 l_1 l_{c2} s_2 \dot{q}_1 & 0 \end{bmatrix}, \tag{9}$$

$$\tau_g(\mathbf{q}) = \begin{bmatrix} -m_1 g l_{c1} s_1 - m_2 g (l_1 s_1 + l_{c2} s_{1+2}) \\ -m_2 g l_{c2} s_{1+2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \tag{10}$$

Example 3.1 (Dynamics of the Acrobot)

You can experiment with the Acrobot dynamics in DRAKE using, e.g.

Open in Colab

3.2 THE CART-POLE SYSTEM

The other model system that we will investigate here is the cart–pole system, in which the task is to balance a simple pendulum around its unstable equilibrium, using only horizontal forces on the cart. Balancing the cart–pole system is used in many introductory courses in control, including 6.003 at MIT, because it can be accomplished with simple linear control (e.g. pole placement) techniques. In this chapter we will consider the full swing–up and balance control problem, which requires a full nonlinear control treatment.

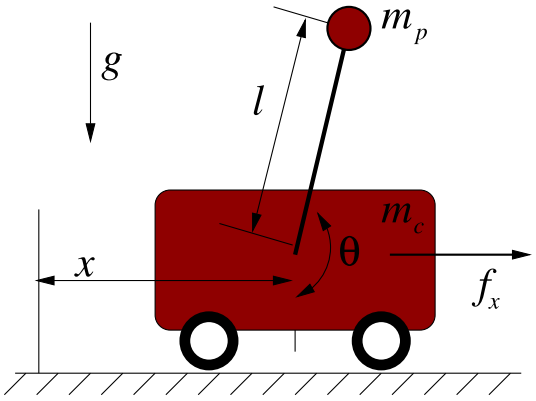


Figure 3.2 – The Cart–Pole system. Click here to see a real robot.

The figure shows our parameterization of the system. x is the horizontal position of the cart, θ is the counter–clockwise angle of the pendulum (zero is hanging straight down). We will use $\mathbf{q} = [x, \theta]^T$, and $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T$. The task is to stabilize the unstable fixed point at $\mathbf{x} = [0, \pi, 0, 0]^T$.

3.2.1 Equations of motion

The kinematics of the system are given by

$$\mathbf{x}_1 = \begin{bmatrix} x \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} x + l\sin\theta \\ -l\cos\theta \end{bmatrix}. \tag{11}$$

The energy is given by

$$T = \frac{1}{2}(m_c + m_p)\dot{x}^2 + m_p\dot{x}\dot{\theta}l\cos\theta + \frac{1}{2}m_p l^2\dot{\theta}^2 \tag{12}$$

$$U = -m_p g l\cos\theta. \tag{13}$$

The Lagrangian yields the equations of motion:

$$(m_c + m_p)\ddot{x} + m_p l\ddot{\theta}\cos\theta - m_p l\dot{\theta}^2\sin\theta = f_x \tag{14}$$

$$m_p l\ddot{x}\cos\theta + m_p l^2\ddot{\theta} + m_p g l\sin\theta = 0 \tag{15}$$

In standard, manipulator equation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

using $\mathbf{q} = [x, \theta]^T$, $\mathbf{u} = f_x$, we have:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} m_c + m_p & m_p l\cos\theta \\ m_p l\cos\theta & m_p l^2 \end{bmatrix}, \quad \mathbf{C}(\mathbf{q},\dot{\mathbf{q}}) = \begin{bmatrix} 0 & -m_p l\dot{\theta}\sin\theta \\ 0 & 0 \end{bmatrix},$$

$$\tau_g(\mathbf{q}) = \begin{bmatrix} 0 \\ -m_p g l\sin\theta \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

In this case, it is particularly easy to solve directly for the accelerations:

$$\ddot{x} = \frac{1}{m_c + m_p\sin^2\theta}\left[f_x + m_p\sin\theta(l\dot{\theta}^2 + g\cos\theta)\right] \tag{16}$$

$$\ddot{\theta} = \frac{1}{l(m_c + m_p\sin^2\theta)}\left[-f_x\cos\theta - m_p l\dot{\theta}^2\cos\theta\sin\theta - (m_c + m_p)g\sin\theta\right] \tag{17}$$

In some of the analysis that follows, we will study the form of the equations of motion, ignoring the details, by arbitrarily setting all constants to 1:

$$2\ddot{x} + \ddot{\theta}\cos\theta - \dot{\theta}^2\sin\theta = f_x \tag{18}$$

$$\ddot{x}\cos\theta + \ddot{\theta} + \sin\theta = 0. \tag{19}$$

Example 3.2 (Dynamics of the Cart-Pole System)

You can experiment with the Cart–Pole dynamics in DRAKE using, e.g.

Open in Colab

3.3 QUADROTORS

Quadrotors have become immensely popular over the last few years –– advances in outrunner motors from the hobby community made them powerful, light–weight, and inexpensive! They are strong enough to carry an interesting payload (e.g. of sensors

for mapping / photographing the environment), but dynamic enough to move relatively quickly. The most interesting dynamics start showing up at higher speeds, when the propellers start achieving lift from the airflow across them due to horizontal motion, or when they are close enough to the ground to experience significant ground–effect, but we won't try to capture those effects (yet) here.

When the quadrotor revolution started to happen, I predicted that it would be followed quickly by a realization that fixed–wing vehicles are better for most applications. Propellers are almost optimally efficient for producing thrust -- making quadrotors very efficient for hovering -- but to be efficient in forward flight you probably want to have an airfoil. Wings are a really good idea! But I was wrong -- quadrotors have completely dominated fixed–wings for commercial UAVs. Perhaps it's only because they are easier to control? I suspect that as the field matures and achieving core functionality is not the primary obstacle, then people will eventually start worrying again about efficiency (a bit like we think about efficiency for automobiles).

3.3.1 The Planar Quadrotor

We can get started with an extremely simple model of a quadrotor that is restricted to live in the plane. In that case, it actually only needs two propellers, but calling it a "birotor" doesn't have the same ring to it. The equations of motion are almost trivial, since it is only a single rigid body, and certainly fit into our standard manipulator equations:

$$m\ddot{x} = -(u_1 + u_2)\sin\theta, \tag{20}$$
$$m\ddot{y} = (u_1 + u_2)\cos\theta - mg, \tag{21}$$
$$I\ddot{\theta} = r(u_1 - u_2) \tag{22}$$

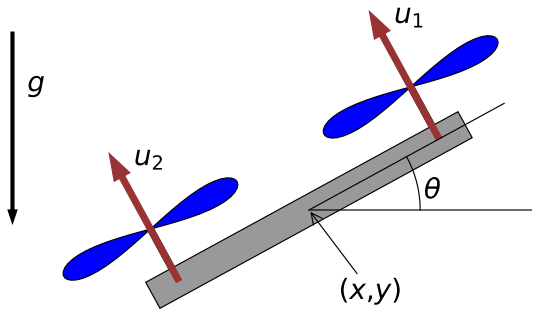


Figure 3.3 – The Planar Quadrotor System (which we also refer to in the code as "Quadrotor2D", to keep it next to "Quadrotor" in the directory listing). The model parameters are mass, m, moment of inertia, I, and the distance from the center to the base of the propeller, r.

3.3.2 The Full 3D Quadrotor

The dynamics of the 3D Quadrotor follow similarly. The only complexity comes from handling the 3D rotations. The code implementing the dynamics of the `QuadrotorPlant` example in Drake is almost as readable as a LaTeX derivation. The most interesting feature of this model that we include the *moment* produced by the

rotating propellers. Interestingly, without these moments, the system linearized about the hovering configuration is actually not controllable.

The `QuadrotorPlant` example implements the simple model. But often you may want to add extra features to the quadrotor model. For instance, you might wish to add collision dynamics to land on the ground or to bounce off obstacles, or you might want to add a pendulum to the quadrotor to balance, or a suspended payload hanging off the bottom. In these cases, rather than implement the equations by hand, it is much more convenient to use Drake's main physics engine (in `MultibodyPlant`). We just need to manually wire the `Propeller` forces into the `Diagram` (because `Propeller` is not a concept supported in URDF nor SDF just yet). I've implemented both versions side-by-side in the notebook.



Note that by default, the orientation of each floating body in `MultibodyPlant` is represented by a unit quaternion. If you were to try to linearize the model using the quaternion representation of state, without accounting for unit norm constraint, the linearization of this model is also uncontrollable. To avoid this for now, I have manually added a "roll-pitch-yaw" floating base to the model. This keeps linearization simple but does introduce a singularity at "gimble lock". I've tried to mostly avoid dealing with 3D rotations throughout these notes, but my notes on manipulation cover 3D rotations in a bit more detail.

3.4 Balancing

For both the Acrobot and the Cart-Pole systems, we will begin by designing a linear controller which can balance the system when it begins in the vicinity of the unstable fixed point. To accomplish this, we will linearize the nonlinear equations about the fixed point, examine the controllability of this linear system, then using linear quadratic regulator (LQR) theory to design our feedback controller.

3.4.1 Linearizing the manipulator equations

Although the equations of motion of both of these model systems are relatively tractable, the forward dynamics still involve quite a few nonlinear terms that must be considered in any linearization. Let's consider the general problem of linearizing a system described by the manipulator equations.

We can perform linearization around a fixed point, $(\mathbf{x}^*, \mathbf{u}^*)$, using a Taylor expansion:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) \approx \mathbf{f}(\mathbf{x}^*, \mathbf{u}^*) + \left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} (\mathbf{x} - \mathbf{x}^*) + \left[\frac{\partial \mathbf{f}}{\partial \mathbf{u}}\right]_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} (\mathbf{u} - \mathbf{u}^*)$$

Let us consider the specific problem of linearizing the manipulator equations around a (stable or unstable) fixed point. In this case, $\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*)$ is zero, and we are left with the standard linear state-space form:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\mathbf{q})\left[\tau_g(\mathbf{q}) + \mathbf{B}(\mathbf{q})\mathbf{u} - \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}\right] \end{bmatrix}, \tag{24}$$

$$\approx \mathbf{A}_{lin}(\mathbf{x} - \mathbf{x}^*) + \mathbf{B}_{lin}(\mathbf{u} - \mathbf{u}^*), \tag{25}$$

where $\mathbf{A}_{lin}$, and $\mathbf{B}_{lin}$ are constant matrices. Let us define $\bar{\mathbf{x}} = \mathbf{x} - \mathbf{x}^*$, $\bar{\mathbf{u}} = \mathbf{u} - \mathbf{u}^*$, and write

$$\dot{\bar{\mathbf{x}}} = \mathbf{A}_{lin}\bar{\mathbf{x}} + \mathbf{B}_{lin}\bar{\mathbf{u}}.$$

Evaluation of the Taylor expansion around a fixed point yields the following, very simple equations, given in block form by:

$$\mathbf{A}_{lin} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{M}^{-1}\frac{\partial \tau_g}{\partial \mathbf{q}} + \sum_j \mathbf{M}^{-1}\frac{\partial \mathbf{B}_j}{\partial \mathbf{q}} u_j & \mathbf{0} \end{bmatrix}_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} \tag{26}$$

$$\mathbf{B}_{lin} = \begin{bmatrix} \mathbf{0} \\ \mathbf{M}^{-1}\mathbf{B} \end{bmatrix}_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} \tag{27}$$

where $\mathbf{B}_j$ is the jth column of $\mathbf{B}$. Note that the term involving $\frac{\partial \mathbf{M}^{-1}}{\partial q_i}$ disappears because $\tau_g + \mathbf{B}\mathbf{u} - \mathbf{C}\dot{\mathbf{q}}$ must be zero at the fixed point. All of the $\mathbf{C}\dot{\mathbf{q}}$ derivatives drop out, too, because $\dot{\mathbf{q}}^* = 0$, and any terms with $\mathbf{C}$ drop out as well, since centripetal and centrifugal forces are zero when velocity is zero. In many cases, including both the Acrobot and Cart-Pole systems (but not the Quadrotors), the matrix $\mathbf{B}(\mathbf{q})$ is a constant, so the $\frac{\partial \mathbf{B}}{\partial \mathbf{q}}$ terms also drop out.

Example 3.3 (Linearization of the Acrobot)

Linearizing around the (unstable) upright point, we have:

$$\left[\frac{\partial \tau_g}{\partial \mathbf{q}}\right]_{\mathbf{x}=\mathbf{x}^*} = \begin{bmatrix} g(m_1 l_{c1} + m_2 l_1 + m_2 l_{c2}) & m_2 g l_{c2} \\ m_2 g l_{c2} & m_2 g l_{c2} \end{bmatrix} \tag{28}$$

The linear dynamics follow directly from these equations and the manipulator form of the Acrobot equations.

Example 3.4 (Linearization of the Cart-Pole System)

Linearizing around the unstable fixed point in this system, we have:

$$\left[\frac{\partial \tau_g}{\partial \mathbf{q}}\right]_{\mathbf{x}=\mathbf{x}^*} = \begin{bmatrix} 0 & 0 \\ 0 & m_p g l \end{bmatrix} \tag{29}$$

Again, the linear dynamics follow simply.

Studying the properties of the linearized system can tell us some things about the (local) properties of the nonlinear system. For instance, having a strictly stable linearization implies local exponential stability of the nonlinear system [3] (Theorem 4.15). It's worth noting that having an unstable linearization also implies that the

system is locally unstable, but if the linearization is marginally stable then one cannot conclude whether the nonlinear system is asymptotically stable, stable i.s.L., or unstable (only that it is not exponentially stable)[4].

3.4.2 Controllability of linear systems

Definition 3.1 (Controllability) A control system of the form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

is called *controllable* if it is possible to construct an unconstrained input signal, $\mathbf{u}(t)$, $t \in [0, t_f]$, which will move the system from any initial state to any final state in a finite interval of time, $0 < t < t_f$ [5].

For the linear system

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

we can integrate this linear system in closed form, so it is possible to derive the exact conditions of controllability. In particular, for linear systems it is sufficient to demonstrate that there exists a control input which drives any initial condition to the origin.

The special case of non-repeated eigenvalues

Let us first examine a special case, which falls short as a general tool but may be more useful for understanding the intuition of controllability. Let's perform an eigenvalue analysis of the system matrix $\mathbf{A}$, so that:

$$\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i,$$

where λ_i is the ith eigenvalue, and $\mathbf{v}_i$ is the corresponding (right) eigenvector. There will be n eigenvalues for the $n \times n$ matrix $\mathbf{A}$. Collecting the (column) eigenvectors into the matrix $\mathbf{V}$ and the eigenvalues into a diagonal matrix $\mathbf{\Lambda}$, we have

$$\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}.$$

Here comes our primary assumption: let us assume that each of these n eigenvalues takes on a distinct value (no repeats). With this assumption, it can be shown that the eigenvectors $\mathbf{v}_i$ form a linearly independent basis set, and therefore $\mathbf{V}^{-1}$ is well–defined.

We can continue our eigenmodal analysis of the linear system by defining the modal coordinates, $\mathbf{r}$, with:

$$\mathbf{x} = \mathbf{V}\mathbf{r}, \quad \text{or} \quad \mathbf{r} = \mathbf{V}^{-1}\mathbf{x}.$$

In modal coordinates, the dynamics of the linear system are given by

$$\dot{\mathbf{r}} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}\mathbf{r} + \mathbf{V}^{-1}\mathbf{B}\mathbf{u} = \mathbf{\Lambda}\mathbf{r} + \mathbf{V}^{-1}\mathbf{B}\mathbf{u}.$$

This illustrates the power of modal analysis; in modal coordinates, the dynamics diagonalize yielding:

$$\dot{r}_i = \lambda_i r_i + \sum_j \beta_{ij} u_j, \quad \beta = \mathbf{V}^{-1}\mathbf{B}.$$

Now the concept of controllability becomes clear. Input j can influence the dynamics in modal coordinate i if and only if $\beta_{ij} \neq 0$. In the special case of non–repeated eigenvalues, having control over each individual eigenmode is sufficient to (in finite time) regulate all of the eigenmodes[5]. Therefore, we say that the system is controllable if and only if

$$\forall i, \exists j \text{ such that } \beta_{ij} \neq 0.$$

A general solution

▶ Included only for completeness. Click to expand the details.

Controllability vs. underactuated

Analysis of the controllability of both the Acrobot and Cart–Pole systems reveals that the linearized dynamics about the upright are, in fact, controllable. This implies that the linearized system, if started away from the zero state, can be returned to the zero state in finite time. This is potentially surprising – after all the systems are underactuated. For example, it is interesting and surprising that the Acrobot can balance itself in the upright position without having a shoulder motor.

The controllability of these model systems demonstrates an extremely important, point: *An underactuated system is not necessarily an uncontrollable system.* Underactuated systems cannot follow arbitrary trajectories, but that does not imply that they cannot arrive at arbitrary points in state space. However, the trajectory required to place the system into a particular state may be arbitrarily complex.

The controllability analysis presented here is for linear time–invariant (LTI) systems. A comparable analysis exists for linear time–varying (LTV) systems. We will even see extensions to nonlinear systems; although it will often be referred to by the synonym of "reachability" analysis.

Stabilizability of a linear system

Closely related to controllability is the notion of *stabilizability*. For linear systems, stabilizability is a strictly weaker condition than controllability (a system can be stabilizable but not controllable, but not the other way around).

Definition 3.2 (Stabilizability of a linear system) A control system of the form

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

is called *stabilizable* if it is possible to construct an unconstrained input signal, $\mathbf{u}(t)$, $t \in [0, \infty]$, which results in

$$\lim_{t \to \infty} \mathbf{x}(t) = 0.$$

Controllability requires that we arrive at the origin in a finite time, stabilizability allows for asympotitic convergence. Essentially, a system can still be stabilizable if the uncontrollable subspace is naturally stable.

Interestingly, for nonlinear systems the relationship between stabilizability and controllability is much more subtle. In a famous (sometimes misquoted) result from Roger Brockett states that nonlinear controllability does not necessarily imply stabilizability by differentiable control policies[8].

3.4.3 LQR feedback

Controllability and stabilizability tell us that a trajectory to the fixed point exists, but does not tell us which one we should take or what control inputs cause it to occur. Why not? There are potentially infinitely many solutions. We have to pick one.

The tools for controller design in linear systems are very advanced. In particular, as we describe in the linear optimal control chapter, one can easily design an optimal feedback controller for a regulation task like balancing, so long as we are willing to linearize the system around the operating point and define optimality in terms of a quadratic cost function:

$$J(\mathbf{x}_0) = \int_0^\infty \left[\mathbf{x}^T(t)\mathbf{Q}\mathbf{x}(t) + \mathbf{u}^T(t)\mathbf{R}\mathbf{u}(t)\right] dt, \quad \mathbf{x}(0) = \mathbf{x}_0, \mathbf{Q} = \mathbf{Q}^T > 0, \mathbf{R} = \mathbf{R}^T >$$

The linear feedback matrix $\mathbf{K}$ used as

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t),$$

is the so-called optimal linear quadratic regulator (LQR). Even without understanding the detailed derivation, we can quickly become practitioners of LQR. DRAKE provides a function,

```
K = LinearQuadraticRegulator(A, B, Q, R)
```

for linear systems. It also provides a version

```
controller = LinearQuadraticRegulator(system, context, Q, R)
```

that will linearize the system for you around an equilibrium and return the controller (in the original coordinates). Therefore, to use LQR, one simply needs to define the symmetric positive-definite cost matrices, $\mathbf{Q}$ and $\mathbf{R}$. In their most common form, $\mathbf{Q}$ and $\mathbf{R}$ are positive diagonal matrices, where the entries Q_{ii} penalize the relative errors in state variable x_i compared to the other state variables, and the entries R_{ii} penalize actions in u_i.

Take a moment to appreciate this. If the linearized system is stabilizable, then for any (positive semi-definite) $\mathbf{Q}$ and (positive definite) $\mathbf{R}$ matrices, LQR will give us a

stabilizing controller. If the system is not stabilizable, then LQR will tell us that no linear controller exists. Pretty amazing!

Example 3.5 (LQR for the Acrobot and Cart-Pole)

Take a minute to play around with the LQR controller for the Acrobot and the Cart–Pole

Open in Colab

Open in Colab

Make sure that you take a minute to look at the code which runs during these examples. Can you set the $\mathbf{Q}$ and $\mathbf{R}$ matrices differently, to improve the performance?

Simulation of the closed–loop response with LQR feedback shows that the task is indeed completed – and in an impressive manner. Oftentimes the state of the system has to move violently away from the origin in order to ultimately reach the origin. Further inspection reveals the (linearized) closed–loop dynamics are in fact non–minimum phase (acrobot has 3 right–half zeros, cart–pole has 1).

Example 3.6 (LQR for Quadrotors)

LQR works essentially out of the box for Quadrotors, if linearized around a nominal fixed point (where the non–zero thrust from the propellers is balancing gravity).

Open in Colab

Open in Colab

or Click here for the animation.

Note that LQR, although it is optimal for the linearized system, is not necessarily the best linear control solution for maximizing basin of attraction of the fixed–point. The theory of *robust control* (e.g., [9]), which explicitly takes into account the differences between the linearized model and the nonlinear model, will produce controllers which outperform our LQR solution in this regard.

3.5 Partial feedback linearization

In the introductory chapters, we made the point that the underactuated systems are not feedback equivalent to $\ddot{q} = \mathbf{u}$. Although we cannot always simplify the full dynamics of the system, it is still possible to linearize a portion of the system dynamics. The technique is called *partial feedback linearization*.

Consider the cart–pole example. The dynamics of the cart are affected by the motions of the pendulum. If we know the model, then it seems quite reasonable to think that we could create a feedback controller which would push the cart in exactly the way necessary to counter–act the dynamic contributions from the pendulum –

thereby linearizing the cart dynamics. What we will see, which is potentially more surprising, is that we can also use a feedback controller for the cart to feedback linearize the dynamics of the passive pendulum joint.

We'll use the term *collocated* partial feedback linearization to describe a controller which linearizes the dynamics of the actuated joints. What's more surprising is that it is often possible to achieve *non–collocated* partial feedback linearization – a controller which linearizes the dynamics of the unactuated joints. The treatment presented here follows from [10].

3.5.1 PFL for the Cart-Pole System

Collocated

Starting from the equations 18 and 19, we have

$$\ddot{\theta} = -\ddot{x}c - s$$
$$\ddot{x}(2 - c^2) - sc - \dot{\theta}^2 s = f_x$$

Therefore, applying the feedback control

$$f_x = (2 - c^2)\ddot{x}^d - sc - \dot{\theta}^2 s \tag{30}$$

results in

$$\ddot{x} = \ddot{x}^d$$
$$\ddot{\theta} = -\ddot{x}^d c - s,$$

which are valid globally.

Look carefully at the resulting equations. Of course, it says that we can impose whatever accelerations we like on the cart. But even the resulting equations of the pole happen to take a nice form here: they have been reduced to the equations of the simple pendulum (without a cart), where the torque input is now given instead by $\ddot{x}c$. It's as if we have a simple pendulum with torque control, except our command is modulated by a $\cos\theta$ term, and this $\cos\theta$ term is fundamental –– it's true that our control authority goes to zero when the pole is horizontal, because no amount of force on the cart in that configuration can act like a torque on the pole.

Non-collocated

Starting again from equations 18 and 19, we have

$$\ddot{x} = -\frac{\ddot{\theta} + s}{c}$$
$$\ddot{\theta}(c - \frac{2}{c}) - 2\tan\theta - \dot{\theta}^2 s = f_x$$

Applying the feedback control

$$f_x = (c - \frac{2}{c})\ddot{\theta}^d - 2\tan\theta - \dot{\theta}^2 s \tag{31}$$

results in

$$\ddot{\theta} = \ddot{\theta}^d$$
$$\ddot{x} = -\frac{1}{c}\ddot{\theta}^d - \tan\theta.$$

Note that this expression is only valid when $\cos\theta \neq 0$. Once again, we know that the force cannot create a torque when the pole is perfectly horizontal. In fact, the controller we have written will "blow-up" -- requesting infinite force at $\cos\theta = 0$; so make sure you saturate the command before you implement it on hardware (or even in simulation). Although it may come as a small consolation, at least we have that $(c - \frac{2}{c})$ never goes to zero; in fact you can check for yourself that $|c - \frac{2}{c}| \geq 1$.

3.5.2 General form

For systems that are trivially underactuated (torques on some joints, no torques on other joints), we can, without loss of generality, reorganize the joint coordinates in any underactuated system described by the manipulator equations into the form:

$$\mathbf{M}_{11}\ddot{\mathbf{q}}_1 + \mathbf{M}_{12}\ddot{\mathbf{q}}_2 = \tau_1, \tag{32}$$
$$\mathbf{M}_{21}\ddot{\mathbf{q}}_1 + \mathbf{M}_{22}\ddot{\mathbf{q}}_2 = \tau_2 + \mathbf{u}, \tag{33}$$

with $\mathbf{q} \in \mathbb{R}^n$, $\mathbf{q}_1 \in \mathbb{R}^l$, $\mathbf{q}_2 \in \mathbb{R}^m$, $l = n - m$. $\mathbf{q}_1$ represents all of the passive joints, and $\mathbf{q}_2$ represents all of the actuated joints, and the $\tau = \tau_g - \mathbf{C}\dot{\mathbf{q}}$ terms capture all of the Coriolis and gravitational terms, and

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} \\ \mathbf{M}_{21} & \mathbf{M}_{22} \end{bmatrix}.$$

Fortunately, because $\mathbf{M}$ is uniformly (e.g. for all $\mathbf{q}$) positive definite, $\mathbf{M}_{11}$ and $\mathbf{M}_{22}$ are also positive definite, by the Schur complement condition for positive definiteness.

Collocated linearization

Performing the same substitutions into the full manipulator equations, we get:

$$\ddot{\mathbf{q}}_1 = \mathbf{M}_{11}^{-1}\left[\tau_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2\right] \tag{34}$$
$$(\mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12})\ddot{\mathbf{q}}_2 - \tau_2 + \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\tau_1 = \mathbf{u} \tag{35}$$

It can be easily shown that the matrix $(\mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12})$ is invertible[10]; we can see from inspection that it is symmetric. PFL follows naturally, and is valid globally.

Non-collocated linearization

$$\ddot{\mathbf{q}}_2 = \mathbf{M}_{12}^{+}\left[\tau_1 - \mathbf{M}_{11}\ddot{\mathbf{q}}_1\right] \tag{36}$$
$$(\mathbf{M}_{21} - \mathbf{M}_{22}\mathbf{M}_{12}^{+}\mathbf{M}_{11})\ddot{\mathbf{q}}_1 - \tau_2 + \mathbf{M}_{22}\mathbf{M}_{12}^{+}\tau_1 = \mathbf{u} \tag{37}$$

where $\mathbf{M}_{12}^+$ is a Moore–Penrose pseudo–inverse. This inverse provides a unique solution when the rank of $\mathbf{M}_{12}$ equals l, the number of passive degrees of freedom in the system (it cannot be more, since the matrix only has l rows). The rank condition in this context is sometimes called the property of "Strong Inertial Coupling". It is state dependent; in the cart–pole example above $\mathbf{M}_{12} = \cos\theta$ and drops rank exactly when $\cos\theta = 0$. A system has Global Strong Inertial Coupling if it exhibits Strong Inertial Coupling in every state.

Task-space partial feedback linearization

In general, we can define some combination of active and passive joints that we would like to control. This combination is sometimes called a "task space". Consider an output function of the form,

$$\mathbf{y} = \mathbf{h}(\mathbf{q}),$$

with $\mathbf{y} \in \mathbb{R}^p$, which defines the task space. Define $\mathbf{H}_1 = \frac{\partial \mathbf{h}}{\partial \mathbf{q}_1}$, $\mathbf{H}_2 = \frac{\partial \mathbf{h}}{\partial \mathbf{q}_2}$, $\mathbf{H} = [\mathbf{H}_1, \mathbf{H}_2]$.

Theorem 3.1 - Task Space PFL

If the actuated joints are commanded so that

$$\ddot{\mathbf{q}}_2 = \bar{\mathbf{H}}^+ \left[\ddot{\mathbf{y}}^d - \dot{\mathbf{H}}\dot{\mathbf{q}} - \mathbf{H}_1 \mathbf{M}_{11}^{-1} \tau_1 \right], \tag{38}$$

where $\bar{\mathbf{H}} = \mathbf{H}_2 - \mathbf{H}_1 \mathbf{M}_{11}^{-1} \mathbf{M}_{12}$. and $\bar{\mathbf{H}}^+$ is the right Moore–Penrose pseudo–inverse,

$$\bar{\mathbf{H}}^+ = \bar{\mathbf{H}}^T (\bar{\mathbf{H}} \bar{\mathbf{H}}^T)^{-1},$$

then we have

$$\ddot{\mathbf{y}} = \ddot{\mathbf{y}}^d. \tag{39}$$

subject to

$$\operatorname{rank}\left(\bar{\mathbf{H}}\right) = p, \tag{40}$$

Proof Sketch. Differentiating the output function we have

$$\dot{\mathbf{y}} = \mathbf{H}\dot{\mathbf{q}}$$
$$\ddot{\mathbf{y}} = \dot{\mathbf{H}}\dot{\mathbf{q}} + \mathbf{H}_1 \ddot{\mathbf{q}}_1 + \mathbf{H}_2 \ddot{\mathbf{q}}_2.$$

Solving 32 for the dynamics of the unactuated joints we have:

$$\ddot{\mathbf{q}}_1 = \mathbf{M}_{11}^{-1} (\tau_1 - \mathbf{M}_{12} \ddot{\mathbf{q}}_2) \tag{41}$$

Substituting, we have

$$\ddot{\mathbf{y}} = \dot{\mathbf{H}}\dot{\mathbf{q}} + \mathbf{H}_1\mathbf{M}_{11}^{-1}(\tau_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2) + \mathbf{H}_2\ddot{\mathbf{q}}_2 \tag{42}$$

$$= \dot{\mathbf{H}}\dot{\mathbf{q}} + \bar{\mathbf{H}}\ddot{\mathbf{q}}_2 + \mathbf{H}_1\mathbf{M}_{11}^{-1}\tau_1 \tag{43}$$

$$= \ddot{\mathbf{y}}^d \tag{44}$$

Note that the last line required the rank condition (40) on $\bar{\mathbf{H}}$ to ensure that the rows of $\bar{\mathbf{H}}$ are linearly independent, allowing $\bar{\mathbf{H}}\bar{\mathbf{H}}^+ = \mathbf{I}$.

In order to execute a task space trajectory one could command

$$\ddot{\mathbf{y}}^d = \mathbf{K}_d(\dot{\mathbf{y}}^d - \dot{\mathbf{y}}) + \mathbf{K}_p(\mathbf{y}^d - \mathbf{y}).$$

Assuming the internal dynamics are stable, this yields converging error dynamics when $\mathbf{K}_p, \mathbf{K}_d > 0$[4], which implies $y(t) \to y^d(t)$. For a position control robot, the acceleration command of (38) suffices. Alternatively, a torque command follows by substituting (38) and (41) into (33).

Example 3.7 (End-point trajectory following with the Cart-Pole system)

Consider the task of trying to track a desired kinematic trajectory with the endpoint of pendulum in the cart–pole system. With one actuator and kinematic constraints, we might be hard–pressed to track a trajectory in both the horizontal and vertical coordinates. But we can at least try to track a trajectory in the vertical position of the end–effector.

Using the task–space PFL derivation, we have:

$$y = h(\mathbf{q}) = -l\cos\theta$$
$$\dot{y} = l\dot{\theta}\sin\theta$$

If we define a desired trajectory:

$$y^d(t) = \frac{l}{2} + \frac{l}{4}\sin(t),$$

then the task–space controller is easily implemented using the derivation above.

The task space derivation above provides a convenient generalization of the partial feedback linearization (PFL) [10], which encompasses both the collocated and non–collocated results. If we choose $\mathbf{y} = \mathbf{q}_2$ (collocated), then we have

$$\mathbf{H}_1 = 0, \mathbf{H}_2 = \mathbf{I}, \dot{\mathbf{H}} = 0, \bar{\mathbf{H}} = \mathbf{I}, \bar{\mathbf{H}}^+ = \mathbf{I}.$$

From this, the command in (38) reduces to $\ddot{\mathbf{q}}_2 = \ddot{\mathbf{q}}_2^d$. The actuator command is then

$$\mathbf{u} = \mathbf{M}_{21}\mathbf{M}_{11}^{-1}(\tau_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2^d) + \mathbf{M}_{22}\ddot{\mathbf{q}}_2^d - \tau_2,$$

and the rank condition (40) is always met.

If we choose $\mathbf{y} = \mathbf{q}_1$ (non-collocated), we have

$$\mathbf{H}_1 = \mathbf{I}, \mathbf{H}_2 = 0, \dot{\mathbf{H}} = 0, \bar{\mathbf{H}} = -\mathbf{M}_{11}^{-1}\mathbf{M}_{12}.$$

The rank condition (40) requires that $\text{rank}(\mathbf{M}_{12}) = l$, in which case we can write $\bar{\mathbf{H}}^+ = -\mathbf{M}_{12}^+\mathbf{M}_{11}$, reducing the rank condition to precisely the "Strong Inertial Coupling" condition described in [10]. Now the command in (38) reduces to

$$\ddot{\mathbf{q}}_2 = \mathbf{M}_{12}^+ \left[\tau_1 - \mathbf{M}_{11}\ddot{\mathbf{q}}_1^d\right] \tag{45}$$

The actuator command is found by substituting (45) into (33), yielding the same results as in [10].

3.6 Swing-up control

3.6.1 Energy shaping

Recall in the last chapter, we used energy shaping to swing up the simple pendulum. This idea turns out to be a bit more general than just for the simple pendulum. As we will see, we can use similar concepts of "energy shaping" to produce swing-up controllers for the acrobot and cart-pole systems. It's important to note that it only takes one actuator to change the total energy of a system.

Although a large variety of swing-up controllers have been proposed for these model systems (c.f. [11], [12], [13], [14], [15], [16], [1], [17]), the energy shaping controllers tend to be the most natural to derive and perhaps the most well-known.

3.6.2 Cart-Pole

Let's try to apply the energy-shaping ideas to the cart-pole system. The basic idea, from [18], is to use collocated PFL to simplify the dynamics, use energy shaping to regulate the pendulum to its homoclinic orbit, then to add a few terms to make sure that the cart stays near the origin. This is a bit surprising... if we want to control the pendulum, shouldn't we use the non-collocated version? Actually, we ultimately want to control both the cart and the pole, and we will build on the collocated version both because it avoids inverting the $\cos\theta$ term that can go to zero and because (when all parameters are set to 1) we were left with a particularly simple form of the equations:

$$\ddot{x} = u \tag{46}$$

$$\ddot{\theta} = -uc - s. \tag{47}$$

The first equation is clearly simple, but even the second equation is exactly the equations of a *decoupled* pendulum, just with a slightly odd control input that is modulated by $\cos\theta$.

Let us regulate the energy of this decoupled pendulum using energy shaping. The energy of the pendulum (a unit mass, unit length, simple pendulum in unit gravity) is given by:

$$E(\mathbf{x}) = \frac{1}{2}\dot{\theta}^2 - \cos\theta.$$

The desired energy, equivalent to the energy at the desired fixed-point, is

$$E^d = 1.$$

Again defining $\tilde{E}(\mathbf{x}) = E(\mathbf{x}) - E^d$, we now observe that

$$\begin{aligned}\dot{\tilde{E}}(\mathbf{x}) =& \dot{E}(\mathbf{x}) = \dot{\theta}\ddot{\theta} + \dot{\theta}s \\ =& \dot{\theta}[-uc - s] + \dot{\theta}s \\ =& -u\dot{\theta}\cos\theta.\end{aligned}$$

Therefore, if we design a controller of the form

$$u = k\dot{\theta}\cos\theta\tilde{E}, \quad k > 0$$

the result is

$$\dot{\tilde{E}} = -k\dot{\theta}^2\cos^2\theta\tilde{E}.$$

This guarantees that $|\tilde{E}|$ is non-increasing, but isn't quite enough to guarantee that it will go to zero. For example, if $\theta = \dot{\theta} = 0$, the system will never move. However, if we have that

$$\int_0^t \dot{\theta}^2(t')\cos^2\theta(t')dt' \to \infty, \quad \text{as } t \to \infty,$$

then we have $\tilde{E}(t) \to 0$. This condition, a version of the LaSalle's theorem that we will develop in our notes on Lyapunov methods, is satisfied for all but the trivial constant trajectories at fixed points.

Now we return to the full system dynamics (which includes the cart). [18] and [19] use the simple pendulum energy controller with an addition PD controller designed to regulate the cart:

$$\ddot{x}^d = k_E\dot{\theta}\cos\theta\tilde{E} - k_p x - k_d\dot{x}.$$

[18] provides a proof of convergence for this controller with some nominal parameters. [11] provides a slightly different controller derived directly from a Lyapunov argument.

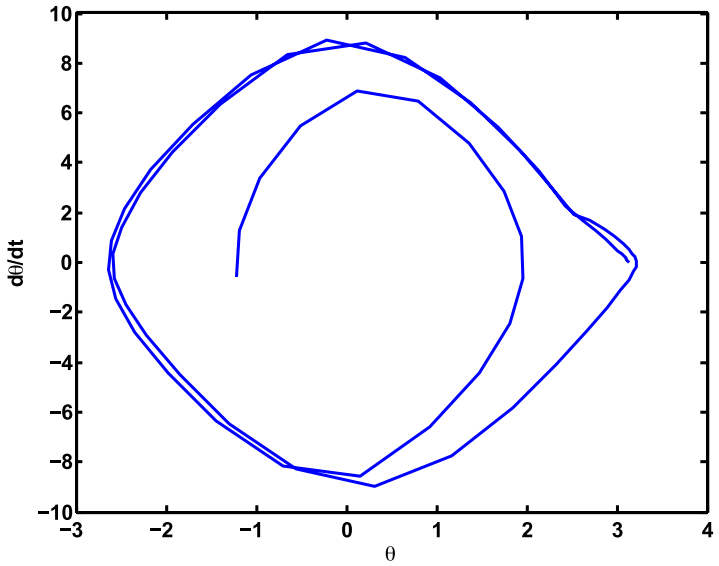


Figure 3.4 – Cart–Pole Swingup: Example phase plot of the pendulum subsystem using energy shaping control. The controller drives the system to the homoclinic orbit, then switches to an LQR balancing controller near the top.

3.6.3 Acrobot

Swing–up control for the acrobot can be accomplished in much the same way. [14] – pump energy. Clean and simple. No proof. Slightly modified version (uses arctan instead of sat) in [20]. Clearest presentation in [19].

Use collocated PFL. ($\ddot{q}_2 = \ddot{q}_2^d$).

$$E(\mathbf{x}) = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}\dot{\mathbf{q}} + U(\mathbf{x}).$$

$$E_d = U(\mathbf{x}^*).$$

$$\bar{u} = \dot{q}_1\tilde{E}.$$

$$\ddot{q}_2^d = -k_1 q_2 - k_2\dot{q}_2 + k_3\bar{u},$$

Extra PD terms prevent proof of asymptotic convergence to homoclinic orbit. Proof of another energy–based controller in [13].

3.6.4 Discussion

The energy shaping controller for swing–up presented here is a pretty faithful representative from the field of nonlinear underactuated control. Typically these control derivations require some clever tricks for simplifying or canceling out terms in the nonlinear equations, then some clever Lyapunov function to prove stability. In these cases, PFL was used to simplify the equations, and therefore the controller design.

We will see another nice example of changing coordinates in the nonlinear equations to make the problem easier when we study "differential flatness" for trajectory

optimization. This approach is perhaps most famous these days for very dynamic trajectory planning with quadrotors.

These controllers are important, representative, and relevant. But clever tricks with nonlinear equations seem to be fundamentally limited. Most of the rest of the material presented in this book will emphasize more general computational approaches to formulating and solving these and other control problems.

3.7 Other model systems

The acrobot and cart–pole systems are just two of the model systems used heavily in underactuated control research. Other examples include:

- Pendubot
- Inertia wheel pendulum
- Furuta pendulum (horizontal rotation and vertical pendulum)
- Hovercraft

3.8 Exercises

Exercise 3.1 (Cart-Pole: Linearization and Balancing)

For this exercise you will work exclusively in this notebook. You will be asked to complete the following steps.

a. Derive the state–space dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ of the cart–pole system.
b. Linearize the dynamics from point (a) around the unstable equilibrium point.
c. Analyze the linearization error for different values of the state $\mathbf{x}$ and the control $\mathbf{u}$.
d. Among the states from point (c), identify which ones are stabilized to the origin by the LQR controller.

Exercise 3.2 (Cart-Poles: Writing URDFs and balancing with LQR)

For this exercise you will work in this notebook. You will complete the following steps.

a. Construct the cart–pole pendulum URDF structure for the single pendulum cart–pole..
b. Extend the single pendulum cart–pole to a double pendulum cart–pole by modifying the URDF model of the robot and testing LQR's ability to control it.

Exercise 3.3 (Controllability of Discrete and Continuous LTI Systems)

a. (2–D grid with discrete double integrator) Consider a discretized grid world where

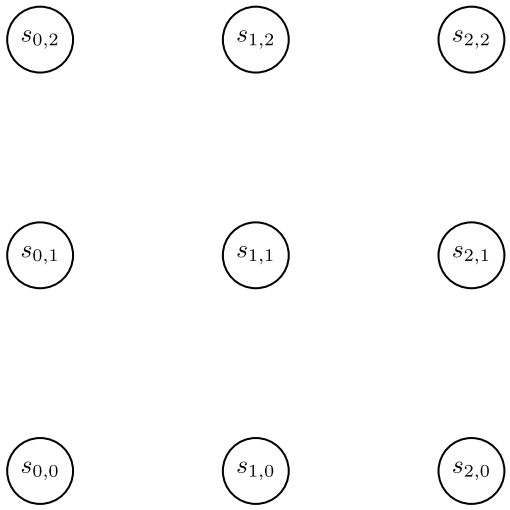
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \text{ and } x_1[n], x_2[n] \in \mathbb{Z}.$$

The dynamics of $x[n]$ is a discrete–time version of double integrator,

$$\mathbf{x}[n+1] = \begin{bmatrix} x_1[n] + x_2[n] \\ x_2[n] + u[n] \end{bmatrix},$$

where the control input $u[n] \in \mathbb{Z}$ is also an integer.

1. Suppose we represent the discretized double integrator dynamics on the following graph, where the nodes $s_{i,j}$ represent state vector values ($\mathbf{x} = s_{i,j}$ means $x_1 = i, x_2 = j$) and edges are state transitions defined by a control law. In the following truncated graph, draw the edges when $u = -1$, $u = 0$, and $u = 1$. (Please draw edges for all the nine nodes in the truncated graph. If a node trainsitions to another node outside the graph, please draw the destination node as well.)



2. Suppose we start from $s_{0,0}$, construct a control input sequence ($u[n]$ could be a function of time step and can take any arbitrary intger values) to reach an arbitrary state $s_{i,j}$ that satisfies $j > i > 0$. What would be the minimal number of time steps needed to reach any $s_{i,j}$ that satisfies $j > i > 0$? Show that the number you've chosen is indeed minimum.
3. Now consider a more general case, from an arbitrary initial state s_{i_1,j_1}, can you find a control input sequence $u[n]$ with the same number of time steps as in (2) that allows the state to reach any final state s_{i_2,j_2}? Also show that the number you've chosen is minimum.

b. 1. Considering linear systems

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

are the following linear systems controllable?

$$\mathbf{A}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \qquad \mathbf{A}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix};$$

$$\mathbf{A}_3 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \mathbf{B}_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \qquad \mathbf{A}_4 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}, \mathbf{B}_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

You should only need the conditions described in the definition of controllability to explain your conclusion. You can also use the more general tools found in this collapsable section; these will prove especially for the $(\mathbf{A}_4, \mathbf{B}_4)$ matrix pair.

2. Consider the two systems, $(\mathbf{A}_3, \mathbf{B}_3)$ and $(\mathbf{A}_4, \mathbf{B}_4)$. Can you make a conclusion whether these two systems are underactuated? (Note that the definition of underactuation requires the system to be interpreted as a second-order system, i.e., $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T$)

Exercise 3.4 (Comparison between stability of nonlinear system and linearization)

In this problem, we use phase portraits to examine how linearization technique helps with local stability analysis for nonlinear systems and its limitations using the nonlinear pendulum with zero torque as an example. The pendulum parameters are chosen as $m = 1$, $l = 1$, $g = 9.81$. Follow the exercises below and complete the graphical analysis for equilibrium points $\mathbf{x}^* = [0, 0]^T$, $\mathbf{x}^* = [\pi, 0]^T$, which will guide you to studying the systems. When you are done, populate this table:

	sign $(Re(\lambda_1))$	sign $(Re(\lambda_2))$	stable i.s.L.	Asymp. Stable	Exp. Stable
Nonlinear $b = 0$, $\mathbf{x}^* = [0, 0]^T$	N/A	N/A			
Nonlinear $b = 0$, $\mathbf{x}^* = [\pi, 0]^T$	N/A	N/A			
Linearization $b = 0$, $\mathbf{x}^* = [0, 0]^T$					
Linearization $b = 0$, $\mathbf{x}^* = [\pi, 0]^T$					
Nonlinear $b = 1$, $\mathbf{x}^* = [0, 0]^T$	N/A	N/A			

Nonlinear $b = 1$, $\mathbf{x}^* = [\pi, 0]^T$	N/A	N/A			
Linearization $b = 1$, $\mathbf{x}^* = [0, 0]^T$					
Linearization $b = 1$, $\mathbf{x}^* = [\pi, 0]^T$					

a. Let's first look at an undamped pendulum $b = 0$.
   1. Using this notebook as an example, draw the phase portrait using `plot_2d_phase_portrait` function around equilibrium $\mathbf{x}^*$. Is the nonlinear undamped pendulum stable i.s.L.? asymptotically stable? exponentially stable?
   2. Derive the linearization of the undamped nonlinear pendulum around equilibrium $\mathbf{x}^*$. Compute the eigenvalues of the Jacobian $\left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}$, what are the signs of the real parts of these two eigenvalues, $\text{sign}(Re(\lambda_1))$, $\text{sign}(Re(\lambda_2))$?
   3. Using phase portrait, is the linearized system $\dot{\mathbf{x}} = \left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}(\mathbf{x} - \mathbf{x}^*)$ stable i.s.L.? exponentially stable?

b. Now let's consider a pendulum with damping coefficient $b = 1$.
   1. Using this notebook as an example, draw the phase portrait using `plot_2d_phase_portrait` function around equilibrium $\mathbf{x}^*$. Is the nonlinear damped pendulum stable i.s.L.? asymptotically stable? exponentially stable?
   2. Derive the linearization of the damped nonlinear pendulum around equilibrium $\mathbf{x}^*$. Compute the eigenvalues of the Jacobian $\left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}$, what are the signs of the real parts of these two eigenvalues, $\text{sign}(Re(\lambda_1))$, $\text{sign}(Re(\lambda_2))$?
   3. Using phase portrait, is the linearized system $\dot{\mathbf{x}} = \left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}(x - \mathbf{x}^*)$ stable i.s.L.? exponentially stable?

References

1. Richard M. Murray and John Hauser, "A case Study in Approximate Linearization: The Acrobot Example", *Memorandum No. UCB/ERL (unknown)* , April, 1991.

2. M. W. Spong, "Underactuated Mechanical Systems", *Control Problems in Robotics and Automation* , 1997.

3. Hassan K. Khalil, "Nonlinear Systems", Prentice Hall , December, 2001.

4. Jean–Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall , October, 1990.

5. Katsuhiko Ogata, "Modern Control Engineering", Prentice Hall Incorporated , August, 1996.

6. Gilbert Strang, "Introduction to Linear Algebra", Wellesley–Cambridge Press , October, 1998.

7. Chi–Tsong Chen, "Linear System Theory and Design", Oxford University Press , Sept 10, 1998.

8. R.W. Brockett, "Asymptotic stability and feedback stabilization", *Differential Geometric Control Theory* , pp. 181–191, 1983.

9. Kemin Zhou and John C. Doyle, "Essentials of Robust Control", Prentice Hall , 1997.

10. Mark Spong, "Partial feedback linearization of underactuated mechanical systems", *Proceedings of the IEEE International Conference on Intelligent Robots and Systems* , vol. 1, pp. 314–321, September, 1994.

11. Isabelle Fantoni and Rogelio Lozano, "Non–linear Control for Underactuated Mechanical Systems", Springer–Verlag , 2002.

12. Araki and N.; Okada and M.; Konishi and Y.; Ishigaki and H.;, "Parameter identification and swing–up control of an Acrobot system", *International Conference on International Technology*, 2005.

13. Xin Xin and M. Kaneda, "New analytical results of the energy based swinging up control of the Acrobot", *Proceedings of the 43rd IEEE Conference on Decision and Control (CDC)* , vol. 1, pp. 704 – 709, Dec, 2004.

14. Mark W. Spong, "Swing up control of the Acrobot", *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* , pp. 2356–2361, 1994.

15. Arun D. Mahindrakar and Ravi N. Banavar, "A swing–up of the acrobot based on a simple pendulum strategy", *International Journal of Control*, vol. 78, no. 6, pp. 424–429, April, 2005.

16. M.D. Berkemeier and R.S. Fearing, "Tracking fast inverted trajectories of the underactuated Acrobot", *Robotics and Automation, IEEE Transactions on*, vol. 15, no. 4, pp. 740–750, Aug, 1999.

17. Xu–Zhi Lai and Jin–Hua She and Simon X. Yang and Min Wu, "Stability Analysis and Control Law Design for Acrobots", *Proceedings of the 2006 IEEE International Conference on Robotics and Automation* , May, 2006.

18. Chung Choo Chung and John Hauser, "Nonlinear Control of a Swinging Pendulum", *Automatica*, vol. 31, no. 6, pp. 851–862, June, 1995.

19. Mark W. Spong, "Energy Based Control of a Class of Underactuated Mechanical Systems", *Proceedings of the 1996 IFAC World Congress* , 1996.

20. Mark Spong, "The Swingup Control Problem for the Acrobot", *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49–55, February, 1995.

Previous Chapter Table of contents Next Chapter

Accessibility

© Russ Tedrake, 2024